

Uniform Real-Fiber Certificates

Goal-directed Hermite forms for algebraic witnesses,
exact solution counts, and parametric quantifier elimination

The proposed capability. Turn a nonlinear algebraic existence or uniqueness goal into a finite circuit of polynomial signs. Keep the circuit valid when roots collide, eigenvalues vanish, or the real solution set becomes empty. Close one remaining real parameter by a complete sign-cell cover, and return exact algebraic witnesses when the supported concrete fragment permits it.

Delivered evidence. An executable SymPy producer; a separate, standard-library-only Python replay implementation; retained certificates, negative controls and test logs; proofs of the mathematical rules; and a concrete Lean integration plan.

Trust boundary. This is not an installed Lean tactic. The new checker is not formally verified. No new certificate in this package was checked by Lean's kernel, and no comparison with `grind` or another Lean tactic was run.

Prepared for Vladimir Reshetnikov

Repository audit: 15 September 2026

Forge snapshot: 58ea206bd7ad501b930add568151217bcc7f82a2

Mathematical foundations are classical; the contribution is the capability design, specialization-safe fragment, exact replay organization, and tested implementation.

Contents

1	The capability to add, and the capability not to claim	4
1.1	A repository-relative delta, not a second Forge overview	4
1.2	What has actually been delivered	5
1.3	Relationship to existing real-algebraic research	5
2	Input language and exact semantics	5
2.1	Parameters, fiber variables, and a monic tower	5
2.2	Logical consequences of a count	6
2.3	What the first implementation refuses	6
3	Compile the goal before computing the algebra	6
3.1	The sign-indicator algebra	7
3.2	Tarski queries only on the support	7
4	A quotient basis that survives every specialization	8
4.1	Why an arbitrary small matrix representation is insufficient	9
4.2	Real points, complex points, and multiplicity	9
5	Hermite trace forms count distinct real points	9
5.1	Two genuinely different matrix computations	10
6	Uniform signatures without symbolic pivot assumptions	10
6.1	Characteristic coefficients as polynomial certificates	11
6.2	Replay by Newton identities	11
6.3	An entire square-root count by hand	12
7	The resulting elimination theorem	12
7.1	Completeness and the resource boundary	13
8	Closing one outer real parameter	13
8.1	The cover polynomial	14
8.2	What the certificate stores	14
8.3	Two boundary-only failures	15
8.4	Limits of this closure layer	15
9	Algebraic witnesses rather than larger rational dictionaries	15
9.1	A concrete univariate witness format	15
9.2	A representation gap that more search fuel cannot fix	16

9.3	Proof-only and computational providers	16
10	Concrete algorithms and cost controls	17
10.1	Producer and replay pipelines	17
10.2	Arithmetic cost rather than a misleading speed guarantee	17
10.3	Optimizations with an unchanged acceptance theorem	18
10.4	A safe route beyond literally triangular input	18
11	A concrete Lean integration plan	19
11.1	The proof obligations are modular	19
11.2	Verified library footholds, not invented APIs	20
11.3	The new source bridge must prove equivalence	20
11.4	What successful replay should return	21
11.5	Admission and scheduling algorithms	21
12	A specific Leant/Djex extension	21
12.1	A provider that supplies mathematical data with its property	22
12.2	What this does not solve in the related projects	22
13	Implementation milestones with acceptance gates	22
14	Experiments, controls, and what the numbers mean	23
14.1	Evidence categories	23
14.2	Exact oracle design and common-mode risks	23
14.3	Representative symbolic certificates	24
14.4	Ablations that demonstrate a capability distinction	25
14.5	Mutation coverage	25
14.6	Standalone replay and retained corrections	25
15	Reproducing and extending the package	26
16	Conclusion	27
A	Certificate schema and implementation map	27
B	Research extensions kept separate from current acceptance	28
	References	29

1 The capability to add, and the capability not to claim

Consider the following family of goals:

$$\forall t \in \mathbb{R}, \quad t \geq 0 \implies \exists! x \in \mathbb{R}, \quad x > 0 \wedge x^5 - x = t. \quad (1)$$

There are several ways to prove this particular theorem by hand. The issue for a tactic is not that the theorem needs a mysterious new argument. It is that a useful *generic worker* should discover algebraic existence, count admissible roots, distinguish a unique root from several roots, and handle the exceptional parameter values without being given the correct witness expression.

This article proposes a worker called *Real Fibers*. It takes a finite algebraic family together with a Boolean condition on its points. It returns an exact, quantifier-free *count circuit* in the parameters. Existence becomes “the count is positive”; uniqueness becomes “the count is one”; a universal property of the fiber becomes “the count of violating points is zero”. For one outer real parameter, a second worker closes the circuit over the entire real line. A third, smaller worker returns an exact algebraic-real witness for a concrete univariate problem.

The central engineering choice is deliberately restrictive: begin with *monic triangular equations*. Their quotient has a fixed, explicit basis at every parameter value. This avoids the unsound but tempting step of computing a basis over a rational-function field and silently using it at parameters where its denominators vanish. Within this fragment, multiple roots are allowed and no discriminant-nonzero assumption is needed.

1.1 A repository-relative delta, not a second Forge overview

The audit used Forge’s README, implementation-status ledger, merged nonlinear section, provenance appendix, certificate contracts, and the dedicated Leant/Djex review at the snapshot on the cover. The related READMEs were inspected as well. These sources describe twenty-seven submissions in three rounds, so merely proposing more induction, SOS, integer projection, analytic enclosures, or finite-state closure would largely repeat existing work [1, 2, 3].

Table 1 states the narrower distinction. This is not a claim to have read every line of every archived submission. It is a comparison with the audited merged design and its explicit capability ledger; it is also not a claim that the underlying real-algebraic mathematics is new.

Already in Forge	What this article adds	Why it is different
Square-factor/Sturm interval certificates	Uniform counts for Boolean-constrained real fibers	Counts roots under parameter specialization; not just nonnegativity on one rational interval.
Affine, lattice, residue, polynomial witnesses	Algebraic root descriptors and count-based existence	A witness need not be a rational polynomial or rational expression.
Mixed-strictness polyhedral projection	Finite nonlinear-fiber elimination	Nonlinear equations define the witnesses; sign correlations are preserved.
Quantitative probabilistic workers	Cardinalities of real algebraic solution sets	The quantity is a distinct-root count, not probability, expectation, or a path bound.
Source-owned evidence in Leant/Djex	A refined algebraic provider with a count precondition	A concrete new producer of typed witness obligations, not a new generic receipt architecture.

Table 1: The proposed addition relative to the audited Forge materials.

Forge’s existing univariate worker is important prior work: it already checks a square-factor identity and a signed-remainder chain to establish nonnegativity on a rational interval. This article does not present Sturm sequences as a new Forge idea. The new object is a *uniform counting certificate*, followed by a complete outer-parameter cover and, where supported, a witness selector.

1.2 What has actually been delivered

The package implements the polynomial admission check, two independent sign-indicator constructions, Hermite-matrix production and replay, characteristic-polynomial replay, rational specialization of count circuits, one-parameter quantified closure, and concrete univariate algebraic witnesses. It includes generated evidence and a replay command that disables site packages and forbids importing the producer.

The Python implementation uses exact rational arithmetic. Its producer and replay perform substantially different computations in the matrix lane, but they are not fully independent: they share the source schema and sparse-polynomial representation. The outer producer also reuses the root-product assembler and verdict evaluator. These are disclosed common-mode risks, not details hidden behind the word “independent”.

No Lean executable was available for this work. The article therefore supplies no purportedly verified Lean tactic and no uncompiled theorem file masquerading as a checker. The proposed Lean interfaces later in the article are design notation. Forge itself records successful elaboration of some earlier core files; that historical fact must not be confused with verification of the new algorithms here [1].

1.3 Relationship to existing real-algebraic research

Hermite trace forms and Tarski queries are classical tools for real root counting. Parametric Hermite methods are developed in work by Le and Safey El Din, and by Gaillard and Safey El Din [4, 5]. Certificate-oriented univariate decision procedures and complete real quantifier elimination have also been formalized in Isabelle/HOL [6, 7]. The present design is neither the first real quantifier-elimination algorithm nor the first verified use of algebraic certificates.

The practical contribution is a small set of choices made together: admit a source fragment with a uniform basis; compile the *actual Boolean goal* rather than every sign condition; verify polynomial identities rather than symbolic pivots; retain zero-sign cases; distinguish count proofs from executable witnesses; and test the whole route on source-bound artifacts. This gives Forge a capability missing from the audited ledger without requiring an immediate port of a general CAD implementation.

2 Input language and exact semantics

2.1 Parameters, fiber variables, and a monic tower

Let $\mathbf{t} = (t_1, \dots, t_k)$ be real parameters and $\mathbf{x} = (x_1, \dots, x_n)$ be the variables to eliminate. Fix positive integers d_i and equations

$$f_i(\mathbf{t}, x_1, \dots, x_i) = x_i^{d_i} + \sum_{j=0}^{d_i-1} a_{ij}(\mathbf{t}, x_1, \dots, x_{i-1}) x_i^j = 0, \quad (2)$$

where every a_{ij} has rational coefficients. The coefficient of $x_i^{d_i}$ is *literally one*, not an expression proved nonzero on a generic region. For a fixed parameter value τ , define

$$\mathcal{Z}_\tau = \{z \in \mathbb{R}^n : f_i(\tau, z) = 0 \text{ for every } i\}.$$

This is a set of distinct real tuples. Algebraic multiplicities are not included in its cardinality.

Let $q_1, \dots, q_m \in \mathbb{Q}[\mathbf{t}, \mathbf{x}]$. An atom compares one q_j with zero using $<, \leq, =, \neq, \geq$, or $>$. A formula F is built with Boolean connectives. The worker's exact target is

$$N_F(\tau) = \#\{z \in \mathcal{Z}_\tau : F(\tau, z)\}. \quad (3)$$

Parameter-only restrictions can be put in F , or kept as assumptions for the outer count circuit. Both interpretations are explicit in the source problem; the certificate does not get to choose one after seeing the result.

2.2 Logical consequences of a count

The useful consequences are

$$\exists \mathbf{x} (\mathbf{f} = 0 \wedge F) \iff N_F > 0, \quad (4)$$

$$\exists! \mathbf{x} (\mathbf{f} = 0 \wedge F) \iff N_F = 1, \quad (5)$$

$$\forall \mathbf{x} (\mathbf{f} = 0 \implies F) \iff N_{\neg F} = 0. \quad (6)$$

The uniqueness statement concerns the whole tuple $\mathbf{x}$. A request for uniqueness of only one projection requires a separate encoding; counting pairs can overcount distinct first coordinates.

The final line is deliberately not $N_F = D$, where $D = \prod_i d_i$. The algebra can have complex roots, repeated roots, or no real roots. An empty real fiber makes the universal implication true. A dimension-based test would get this wrong even for $x^2 + 1 = 0$.

2.3 What the first implementation refuses

The source language excludes variable denominators, nonmonic leading coefficients, positive-dimensional fibers, transcendental equations, integer quantification, and arbitrary dependent source syntax. Refusal is not a proof of falsity. It is an opportunity for Forge's existing controller to try another worker.

For example, $tx = 0$ has one solution when $t \neq 0$ and infinitely many when $t = 0$. Dividing by t destroys the exceptional fiber. The present admission test rejects this source rather than pretending that a generic rank-one quotient works everywhere. By contrast, $(x - t)^4 = 0$ is admitted: it has a uniform four-dimensional quotient but exactly one distinct real root at every parameter value.

The mathematical results below impose no small numerical caps. The prototype does: at most four parameters, four fiber variables, six sign atoms, degree at most twelve in each leading variable, and quotient dimension at most twelve. A later query-support cap can still stop an admitted problem. The experiments exercise at most two parameters and two fiber variables; the larger parser limits are not a claim of equally extensive coverage.

3 Compile the goal before computing the algebra

A general sign-determination algorithm can compute how many points realize each of the 3^m sign vectors in $\{-1, 0, 1\}^m$. A particular proof goal often needs much less information. We first

turn its truth value into a polynomial on this finite sign cube.

3.1 The sign-indicator algebra

Introduce variables $s_1, \dots, s_m$ with relations $s_j^3 = s_j$. Their intended values are $s_j = \text{sgn}(q_j)$. The following identities are exact on the sign cube:

$$\begin{aligned} \mathbf{1}_{s>0} &= \frac{s^2 + s}{2}, & \mathbf{1}_{s<0} &= \frac{s^2 - s}{2}, & \mathbf{1}_{s=0} &= 1 - s^2, \\ \mathbf{1}_{s \neq 0} &= s^2, & \mathbf{1}_{s \geq 0} &= 1 - \frac{s^2 - s}{2}, & \mathbf{1}_{s \leq 0} &= 1 - \frac{s^2 + s}{2}. \end{aligned}$$

If I and J are the indicators of two formulas, Boolean operations become

$$\begin{aligned} I_{\neg F} &= 1 - I, & I_{F \wedge G} &= IJ, & I_{F \vee G} &= I + J - IJ, \\ I_{F \Rightarrow G} &= 1 - I + IJ, & I_{F \Leftrightarrow G} &= 1 - I - J + 2IJ. \end{aligned}$$

After each product, replace a positive exponent a by $1 + ((a - 1) \bmod 2)$. Thus every monomial has exponent vector in $\{0, 1, 2\}^m$. The zero exponent remains zero; in particular, $s^0 = 1$ even when $s = 0$.

Proposition 3.1 (Exact indicator compiler). *The recursive construction yields a polynomial*

$$I_F(\mathbf{s}) = \sum_{e \in \{0,1,2\}^m} c_e \mathbf{s}^e$$

with rational coefficients such that $I_F(\mathbf{s})$ is exactly 1 or 0 according as F is true or false on the sign vector $\mathbf{s}$.

Proof. Check each atomic identity at $-1, 0, 1$. The Boolean identities are the truth tables of the corresponding operations on $\{0, 1\}$. Reduction by $s_j^3 = s_j$ preserves values on the cube. Structural induction on the source formula proves the result. $\square$

The prototype's producer intentionally uses a different construction. For each sign vector satisfying F , it adds the tensor-product Lagrange indicator

$$\prod_{j=1}^m L_{s_j}(z_j), \quad L_0(z) = 1 - z^2, \quad L_{+1}(z) = \frac{z^2 + z}{2}, \quad L_{-1}(z) = \frac{z^2 - z}{2}.$$

The replay side uses the recursive compiler above. The reduced polynomials are equal because these tensor products form the interpolation basis for all functions on the sign cube. This producer is deliberately simple, not the preferred scalable implementation: it enumerates the cube even when the final query support is sparse.

3.2 Tarski queries only on the support

For a finite real zero set Z , define

$$\text{TaQ}(w, Z) = \sum_{z \in Z} \text{sgn}(w(z)).$$

The multiplicativity of the real sign function gives

$$N_F(\mathbf{t}) = \sum_{e: c_e \neq 0} c_e \text{TaQ} \left(\prod_j q_j^{e_j}, Z_t \right). \quad (7)$$

Only the nonzero coefficient support is needed. Let its size be r . A single positivity atom requires two queries rather than three. Two simultaneous positivity atoms require four rather than nine. A constant-true formula needs only the weight 1, irrespective of unused source atoms.

This is an exact elimination of irrelevant queries, not heuristic pruning. The worst case remains $r = 3^m$. A production compiler should use a canonical sparse map or a decision diagram during Boolean compilation, share repeated subformulas, and enforce a support budget before any large matrix computation.

Example 3.2 (Why marginal counts are not enough). On the roots of $x^2 - 1 = 0$, each of $x > 0$ and $-x > 0$ has one solution. Their conjunction has none. Its indicator is

$$\frac{1}{4} (s_1 s_2 + s_1 s_2^2 + s_1^2 s_2 + s_1^2 s_2^2).$$

For $q_1 = x$ and $q_2 = -x$, the four Tarski queries are respectively $-2, 0, 0, 2$, so the answer is zero. Keeping only the two marginal counts loses precisely the correlation required by the goal.

4 A quotient basis that survives every specialization

Set $R_0 = \mathbb{Q}[t]$ and

$$\mathcal{A} = R_0[x_1, \dots, x_n] / (f_1, \dots, f_n).$$

There is no rational-function field in this definition. The candidate basis is the rectangular monomial set

$$\mathcal{B} = \{x_1^{a_1} \cdots x_n^{a_n} : 0 \leq a_i < d_i\}, \quad D = \prod_i d_i. \quad (8)$$

Lemma 4.1 (Monic quotient basis). *For any commutative ring R and a monic polynomial $f \in R[X]$ of degree $d > 0$, the quotient $R[X]/(f)$ is free over R with basis $1, X, \dots, X^{d-1}$.*

Proof. Division by a monic polynomial is possible without dividing any coefficient in R . Repeatedly subtract the appropriate multiple of f to reduce the degree. This proves spanning. For uniqueness, the difference of two degree-less-than- d remainders would equal fh . If $h \neq 0$, the leading coefficient of fh is the leading coefficient of h , because f is monic; hence $\deg(fh) = d + \deg h \geq d$, a contradiction. Thus the difference is zero. $\square$

Theorem 4.2 (Uniform finite free tower). *For equations of the form (2), $\mathcal{B}$ is an R_0 -basis of $\mathcal{A}$. For every $\tau \in \mathbb{R}^k$, its specialized images form an $\mathbb{R}$ -basis of*

$$\mathcal{A}_\tau = \mathbb{R}[\mathbf{x}] / (f_1(\tau), \dots, f_n(\tau)).$$

Polynomial reduction, multiplication matrices, and their traces commute with specialization.

Proof. Construct the quotient one variable at a time. After adjoining $x_1, \dots, x_{i-1}$, equation f_i remains monic in x_i over the preceding quotient ring. Apply Lemma 4.1; multiplying the successive bases yields (8). Specializing the parameters is a ring homomorphism and preserves the leading coefficient 1. The same quotient argument therefore gives exactly the same basis size in every specialized algebra.

Alternatively, verify commutation directly on the normal-form algorithm. Replace each $x_i^{d_i}$ by the negative lower-degree tail of f_i , starting with the largest variable index. Each replacement is a polynomial identity over R_0 , and evaluation at τ preserves it. The specialized normal form is unique by the basis result. Multiplication-matrix entries are coefficients of these normal forms, and traces are finite sums of diagonal entries, so both commute as well. $\square$

The descending-variable rewrite is terminating: a rewrite lowers the exponent of its highest affected variable, and can increase only lower-variable exponents. The induced lexicographic monomial order is well-founded. The prototype also has operational caps because a terminating expansion may still be too large.

4.1 Why an arbitrary small matrix representation is insufficient

A collection of commuting matrices satisfying $f_i(M_1, \dots, M_n) = 0$ is not enough to count *all* roots of the source ideal. It may represent only one component. For $x^2 - 1 = 0$, the one-by-one matrix $[1]$ satisfies the equation but loses the root -1 .

The worker must use the regular representation on the *entire source quotient*, with its proved basis, or provide a separately proved equivalent presentation. A numerical dimension guess, a matrix satisfying the equations, or a one-way ideal inclusion cannot replace this requirement. This is one of the strongest reasons to use the monic tower in the first version: the admission theorem supplies the missing completeness relation without asking a certificate to prove a general Gröbner basis.

4.2 Real points, complex points, and multiplicity

The dimension D counts the complex algebra with multiplicity, not the distinct real solutions. Three simple fibers expose the differences:

Equation	D	$\#\mathcal{Z}$
$x^2 - 1 = 0$	2	2
$x^2 + 1 = 0$	2	0
$x^2 = 0$	2	1

These are not corner cases to exclude. They are ordinary fibers of the admitted language. The next theorem handles all three with one trace-form rule.

5 Hermite trace forms count distinct real points

For $p \in \mathcal{A}$, let M_p denote multiplication by p in the basis $\mathcal{B}$. For a weight w , form the symmetric matrix

$$H_w(i, j) = \text{Tr}(M_{wb_i b_j}), \quad b_i, b_j \in \mathcal{B}. \quad (9)$$

Its entries are rational polynomials in the parameters. The signature of a real symmetric matrix is the number of positive eigenvalues minus the number of negative eigenvalues, with multiplicities; zero eigenvalues contribute zero.

The following is the classical Hermite trace-form principle underlying this worker [4, 5]. We include its algebraic proof because the treatment of repeated roots is essential to the design, not an optional optimization.

Theorem 5.1 (Hermite signature, including nonreduced fibers). *For every admitted source and every real parameter tuple τ ,*

$$\text{sig } H_w(\tau) = \sum_{z \in \mathcal{Z}_\tau} \text{sgn}(w(\tau, z)). \quad (10)$$

Each distinct real point appears once, even when its algebraic multiplicity exceeds one.

Proof. Fix the parameters and write $A = \mathcal{A}_\tau$. It is a finite-dimensional commutative real algebra. Such an algebra is Artinian and decomposes as a product of local Artinian algebras. Their residue fields are finite extensions of $\mathbb{R}$, hence are $\mathbb{R}$ or $\mathbb{C}$. Multiplication and the trace pairing respect this product decomposition, so signatures add over its factors.

Consider one local factor with maximal ideal $\mathfrak{m}$ and residue field K . The ideal $\mathfrak{m}$ is nilpotent. Filter the algebra by its powers. Each successive quotient is a finite-dimensional K -vector space, and multiplication by a acts there as multiplication by its residue $\bar{a} \in K$. If the sum of these K -dimensions is $\ell > 0$, additivity of trace along the filtration gives

$$\mathrm{Tr}_{\mathbb{R}}(M_a) = \ell \mathrm{Tr}_{K/\mathbb{R}}(\bar{a}).$$

Consequently the weighted bilinear form on this factor is

$$(a, b) \longmapsto \ell \mathrm{Tr}_{K/\mathbb{R}}(\bar{w} \bar{a} \bar{b}).$$

It vanishes whenever one argument lies in $\mathfrak{m}$, so nilpotent directions contribute only zero eigenvalues.

For $K = \mathbb{R}$, the remaining form is the one-dimensional form $\ell w(z)uv$. Its signature is $\mathrm{sgn} w(z)$, since ℓ is positive. The local length changes the magnitude, not the sign or the number of nonzero directions.

For $K = \mathbb{C}$, write $\bar{w} = a + bi$ and use the real basis $1, i$. Apart from the positive scalar ℓ , the matrix is

$$2 \begin{pmatrix} a & -b \\ -b & -a \end{pmatrix}.$$

When $\bar{w} \neq 0$ its determinant is negative, so it has one positive and one negative eigenvalue. When $\bar{w} = 0$ it is zero. In both cases its signature is zero.

The real residue-field factors are in bijection with the distinct real common zeros of the source equations. Summing their contributions proves (10). The uniform-basis theorem identifies the computed specialization of (9) with this trace form. $\square$

5.1 Two genuinely different matrix computations

The producer builds multiplication matrices using a SymPy Gröbner normal form. It forms H_1 and uses

$$H_w = H_1 M_w. \tag{11}$$

The identity follows by expanding $w b_j$ in the basis and using linearity of trace. Although the right side is not visibly symmetric, it equals the symmetric trace matrix.

The replay side does not trust M_w , does not call a Gröbner engine, and does not reuse (11). It constructs each entry by direct triangular substitution of $w b_i b_j b_\ell$ and sums the coefficient of b_ℓ over ℓ . This is the trace in (9). Exact equality with the supplied matrix is required.

This distinction matters. A producer can make a matrix-layout or normal-form error while still returning a matrix with a plausible spectrum. Checking its characteristic polynomial alone would validate only that wrong matrix. The replay must first bind the matrix to the source quotient.

6 Uniform signatures without symbolic pivot assumptions

A symmetric elimination method can compute a signature over a field, but symbolic pivoting requires branches for zero pivots. Ignoring those branches is especially dangerous here: the source

goals care about precisely the parameter values at which roots coalesce. We use a coefficient-sign formula that includes zeros directly.

6.1 Characteristic coefficients as polynomial certificates

Write

$$\chi_H(\lambda) = \det(\lambda I - H) = \lambda^D + c_1 \lambda^{D-1} + \cdots + c_D, \quad c_0 = 1. \quad (12)$$

For our matrices every c_j belongs to $\mathbb{Q}[\mathbf{t}]$. Let V count sign variations after deleting zero entries. Then define

$$S_H(\mathbf{t}) = V(1, c_1(\mathbf{t}), \dots, c_D(\mathbf{t})) - V((-1)^D, (-1)^{D-1} c_1(\mathbf{t}), \dots, c_D(\mathbf{t})). \quad (13)$$

Only signs are passed to V ; the notation suppresses this conversion.

Lemma 6.1 (Descartes is exact for a real-rooted polynomial). *If a nonzero real polynomial p has only real roots, then its positive-root count, including multiplicity, equals $V(p)$, the sign variations of its descending coefficient list. Its negative-root count equals $V(p(-X))$.*

Proof. Remove a power of X , which does not affect coefficient variations or nonzero roots. Let the remaining degree be d , and let n_+ and n_- be its positive- and negative-root counts. Real-rootedness gives $n_+ + n_- = d$. Descartes' rule gives $n_+ \leq V(p)$ and $n_- \leq V(p(-X))$.

For completeness, $V(p) + V(p(-X)) \leq d$ even when internal coefficients vanish. Compare two consecutive nonzero coefficients whose exponents differ by g . If g is odd, the corresponding pair contributes exactly one variation across the two lists. If g is even, it contributes zero or two, which is at most g . Sum over exponent gaps. The first and last surviving exponents differ by d . Hence the two inequalities from Descartes must both be equalities. $\square$

Theorem 6.2 (Zero-safe signature formula). *For every real symmetric $D \times D$ matrix H , $S_H = \text{sig } H$.*

Proof. The spectral theorem makes χ_H real-rooted. Its positive and negative roots, with multiplicities, are exactly the positive and negative eigenvalues. Apply Lemma 6.1 to χ_H and $\chi_H(-X)$. Trailing zero coefficients represent zero eigenvalues and are deleted by the variation count. No nonvanishing-minor or simple-eigenvalue hypothesis appears. $\square$

A sign-variation routine without the real-rootedness premise is not a general root-counting algorithm. For example, $X^2 - X + 1$ has two variations but no real roots. Here real-rootedness is earned by the *checked symmetric matrix*, not inferred from an arbitrary polynomial supplied by the producer.

6.2 Replay by Newton identities

The producer uses SymPy's characteristic-polynomial routine. The checker instead verifies all D coefficient identities

$$j c_j = - \sum_{i=1}^j c_{j-i} \text{Tr}(H^i), \quad 1 \leq j \leq D. \quad (14)$$

Both sides are compared as exact rational polynomials. Matrix powers and traces are recomputed by the replay implementation.

Lemma 6.3 (Newton replay is sufficient). *Over a commutative $\mathbb{Q}$ -algebra, a list $c_0 = 1, c_1, \dots, c_D$ satisfying (14) is the coefficient list of $\det(\lambda I - H)$.*

Proof. The actual characteristic coefficients satisfy (14). One derivation uses the formal power series $E(z) = \det(I - zH)$. Since $E(0) = 1$, its inverse exists. The adjugate identity and differentiation of the determinant give

$$\frac{E'(z)}{E(z)} = -\operatorname{Tr}(H(I - zH)^{-1}) = -\sum_{i \geq 1} \operatorname{Tr}(H^i) z^{i-1}.$$

Comparing coefficients in E' yields (14). Conversely, the j th identity uniquely determines c_j from the earlier coefficients, since the nonzero integer j is invertible in a $\mathbb{Q}$ -algebra. Induction identifies the supplied coefficients with the actual ones. $\square$

The only divisions in this argument are by positive integers, not by parameter polynomials. At $t = 0$, at a discriminant zero, or at any other specialization, the same identities remain valid. A future fraction-free implementation can clear these fixed denominators; that is a performance change, not a change of acceptance conditions.

6.3 An entire square-root count by hand

For $f = x^2 - t$, use basis $(1, x)$. The three useful trace matrices are

$$H_1 = \begin{pmatrix} 2 & 0 \\ 0 & 2t \end{pmatrix}, \quad H_x = \begin{pmatrix} 0 & 2t \\ 2t & 0 \end{pmatrix}, \quad H_{x^2} = \begin{pmatrix} 2t & 0 \\ 0 & 2t^2 \end{pmatrix}. \quad (15)$$

Their characteristic polynomials are

$$\begin{aligned} \chi_{H_1} &= \lambda^2 - 2(1+t)\lambda + 4t, \\ \chi_{H_x} &= \lambda^2 - 4t^2, \\ \chi_{H_{x^2}} &= \lambda^2 - 2t(1+t)\lambda + 4t^3. \end{aligned}$$

The indicator formulas give

$$N_{x>0} = \frac{1}{2}(\operatorname{sig} H_x + \operatorname{sig} H_{x^2}), \quad N_{x \geq 0} = \operatorname{sig} H_1 + \frac{1}{2} \operatorname{sig} H_x - \frac{1}{2} \operatorname{sig} H_{x^2}.$$

For $t < 0$ both counts are zero. For $t > 0$ both are one. At $t = 0$, H_1 has signature one while the other matrices are zero; the counts are therefore zero and one respectively. The strictness distinction is not recovered by a numerical tolerance. It is a separate exact sign case.

The coefficient $-2(1+t)$ also vanishes at $t = -1$, where the root count does not change. A coefficient-sign decomposition may thus have more cells than the minimal geometric decomposition. Correctness does not require minimality, and the prototype's five-cell square-root cover deliberately retains this extra boundary.

7 The resulting elimination theorem

For every exponent vector in the indicator support, let $w_e = \prod_j q_j^{e_j}$. Compute H_{w_e} and its checked characteristic coefficients. Define

$$\mathcal{C}_F(\mathbf{t}) = \sum_{e: c_e \neq 0} c_e S_{H_{w_e}}(\mathbf{t}). \quad (16)$$

Theorem 7.1 (Uniform finite-fiber elimination). *For every monic triangular source, every Boolean sign formula F , and every real parameter tuple τ ,*

$$\mathcal{C}_F(\tau) = N_F(\tau).$$

The expression on the left is a finite circuit built from signs of rational parameter polynomials, finite sign-variation counts, and rational arithmetic. In particular, every comparison of N_F with an integer is equivalent to a quantifier-free Boolean combination of parameter-polynomial comparisons.

Proof. Proposition 3.1 and finite summation give (7). Theorem 4.2 makes the computed trace matrices valid at every specialization. Theorem 5.1 replaces each Tarski query by its signature. Lemma 6.3 identifies the checked characteristic coefficients, and Theorem 6.2 evaluates the signature by their signs. Substitution yields (16).

For the final assertion, enumerate the three possible signs of each distinct parameter polynomial appearing in the circuit. On each sign assignment the circuit has a fixed rational value, so an integer comparison is either true or false. Form the disjunction of the corresponding sign conditions for the true assignments. Unrealizable assignments cause no difficulty: their conjunctions have no real parameter instances. This is a finite quantifier-free formula, although an explicit expansion may be much larger than the circuit. $\square$

Corollary 7.2 (Meaning of certificate acceptance). *If the source admission check, indicator-support check, trace-matrix checks, and Newton checks succeed, the supplied count circuit is mathematically correct at every real parameter value. No root-separation, nonzero-discriminant, or genericity assumption is needed for this admitted fragment.*

This corollary describes the mathematics that a Lean checker must formalize. Executing its Python analogue is not itself a proof that the Python implementation satisfies the corollary. The tests later in the article give implementation evidence, not a replacement for that formalization.

7.1 Completeness and the resource boundary

Ignoring resource limits, the admitted source class has a finite construction: the basis is finite, the Boolean indicator has at most 3^m terms, and each characteristic polynomial is finite. Thus the specified method computes the exact count circuit for every source in the class. This is a *fragment* completeness result, not a claim to decide all nonlinear real goals.

The delivered program is bounded. It can stop during polynomial expansion, indicator construction, matrix arithmetic, or outer root-product formation. A cap does not imply that the goal is false, that no certificate exists, or that a competing worker would fail. It produces no mathematical verdict. The same qualification applies to a proposed but unaccepted algebraic witness.

8 Closing one outer real parameter

A uniform count circuit is already a useful elimination result, but it may leave a theorem about its parameters. For exactly one real parameter t , the package implements a complete sign-cell cover. This is a small, exact one-dimensional elimination layer; it is not a general multivariate CAD engine.

8.1 The cover polynomial

Collect every nonconstant, nonzero polynomial occurring among the characteristic coefficients used by the count circuit. Add every parameter polynomial in the outer condition. Form the monic squarefree product, equivalently the least common multiple of their monic squarefree parts:

$$P(t) = \text{sqfree} \left(\prod_{g \in \mathcal{G}} g(t) \right). \quad (17)$$

When $\mathcal{G}$ is empty, take $P = 1$. Repeated copies and scalar multiples need not be retained. The checker constructs this polynomial from the already checked circuit and independently supplied outer condition; the producer cannot omit a difficult coefficient from $\mathcal{G}$.

Let the distinct real roots of P be $\alpha_1 < \dots < \alpha_s$. They define $2s + 1$ cells: the point sections $\{\alpha_i\}$ and the open sectors between consecutive roots, including both unbounded tails. Every support-polynomial sign is constant on each sector, by continuity and absence of roots.

8.2 What the certificate stores

The certificate supplies rational open intervals (ℓ_i, u_i) containing the sections. It does not supply a truth table that the replay simply believes. The replay checks

$$\ell_i < u_i < \ell_{i+1}, \quad \text{Count}(P; (\ell_i, u_i)) = 1, \quad s = \text{Count}(P; \mathbb{R}).$$

No finite endpoint may be a root of P . The exact counts use a signed rational remainder sequence constructed by the checker. The final equality establishes coverage: the disjoint intervals already account for every real root of P .

For a sector, choose any rational point between the adjacent isolators; choose a point beyond the outer isolator for a tail. For a section, determine the sign of a support polynomial g as follows. If g is zero or constant, its sign is immediate. Otherwise count its roots in the section isolator. Every root of g is a root of P , and the isolator contains exactly one root of P . Therefore a count of one means $g(\alpha_i) = 0$. A count of zero means g has constant nonzero sign throughout the isolator, so its rational midpoint gives the sign at α_i .

This argument depends on forming P from *all* support polynomials. Without that inclusion, a root of g inside the isolator could be unrelated to α_i . The witness checker in the next section cannot use this shortcut unless it also isolates a suitable union of roots.

Theorem 8.1 (Complete one-parameter sign cover). *An accepted cover determines the exact count-circuit value and outer-condition truth value on every cell of the real line. Consequently a universal outer quantifier is the conjunction of the cell truth values, and an existential outer quantifier is their disjunction.*

Proof. The interval counts, disjointness, and total root count identify all real roots of P in order. No sector contains any support-polynomial root, so every such polynomial has the sign of its rational sector sample. The section procedure above computes the exact signs at each boundary point. The uniform elimination theorem then gives the exact fiber count on each cell; evaluating the outer Boolean condition is exact as well. These cells partition $\mathbb{R}$, proving both quantifier reductions. $\square$

An isolator is a representation of an exact algebraic point, not a numerical approximation substituted for that point. A false universal claim can therefore return an algebraic counterparameter

section. Likewise, a true existential claim may be witnessed only on a section, with no rational parameter satisfying it.

8.3 Two boundary-only failures

The first regression asks whether

$$\forall t \geq 0, \quad \exists! x > 0, \quad x^2 = t.$$

It is false only at $t = 0$. Every open sector of the complete cover makes the implication true: for $t < 0$ the premise fails; for $t > 0$ there is one positive root. Removing the section test produces a false acceptance.

The second asks whether

$$\forall t \in \mathbb{R}, \quad t^2 - 2 \geq 0 \implies \exists! x > 0, \quad x^2 = t^2 - 2.$$

It is false exactly at $t = \pm\sqrt{2}$. Again every open sector passes. Moreover, testing *every rational parameter* would still miss the counterexamples. The retained artifact contains isolating intervals for the algebraic sections and rejects the universal claim there.

The dual positive control is

$$\exists t \in \mathbb{R}, \quad t^2 - 2 = 0 \wedge \#\{x \geq 0 : x^2 = t^2 - 2\} = 1.$$

It is true only at the two algebraic sections. The package checks an existential verdict and records a section as the decisive cell. Thus the cover is used for both negative and positive evidence, not merely for a collection of universal tests.

8.4 Limits of this closure layer

With two or more outer parameters, the prototype still constructs a uniform count circuit and evaluates it at exact rational parameter tuples, but it does not build a complete parameter-space decomposition. Sampling such tuples cannot prove a universal statement. A future higher-dimensional closure module needs a separately certified decomposition or a theorem proving the count-circuit guard from the original hypotheses. It cannot turn the existing rational specialization tests into that theorem.

The one-parameter cover is not promised to be minimal. Characteristic coefficients may vanish without changing the final count. Merging adjacent cells is safe only after checking that the section and both neighboring sectors have the same relevant result. That compression is a prospective optimization, not part of the recorded implementation.

9 Algebraic witnesses rather than larger rational dictionaries

The count theorem establishes existence, but an executable witness is another artifact. These two outputs must not be conflated.

9.1 A concrete univariate witness format

For a rational univariate source $f(x) = 0$, the implemented descriptor is an open rational interval (ℓ, u) containing exactly one distinct real root of f . The source polynomial remains part of the

independent problem, not a replacement chosen by the certificate. The semantic value is “the unique root of this source polynomial in this interval”.

For each source sign atom q , the witness replay computes $g = \gcd(f, q)$. If g has a root in the isolator, it must be the isolated f -root, and q is zero there. Otherwise the replay requires that q have no root anywhere in the isolator and determines its sign at a rational midpoint. If an unrelated q -root remains in the isolator, the certificate is refused with a request for refinement; the checker does not infer a sign from proximity.

Proposition 9.1 (Witness-descriptor soundness). *If the source-root count is one and every atom sign is checked as above, evaluating F to true proves that the uniquely isolated root satisfies the original source formula.*

Proof. The source-root count gives existence and uniqueness within the isolator. A common root of f and q is a root of their greatest common divisor, and conversely. If the common-root count is one, it is the unique f -root in the interval. In the other case, the no- q -root check and continuity fix the atom’s sign throughout the interval. Hence all atom truth values at the isolated root are correct; structural evaluation of F completes the argument. $\square$

Local isolation does not establish that the whole source has only one admissible root. Global uniqueness needs $N_F = 1$ or another global argument. Conversely, $N_F > 0$ does not magically provide a rational interval selector for a multivariate parameterized fiber. The delivered selector is concrete, rational-coefficient, and univariate.

9.2 A representation gap that more search fuel cannot fix

The descriptor for the positive root of $x^2 - 2$ is not a rational witness. For example, the generated interval

$$\frac{41}{29} < \sqrt{2} < \frac{58}{41}$$

is a representation of the root, not a claim that either endpoint equals it.

More generally, no rational function $r(t) \in \mathbb{Q}(t)$ satisfies $r(t)^2 = t$ on a nonempty open interval. Writing $r = p/q$ and clearing denominators would give the polynomial identity $p^2 = tq^2$. Comparing degrees yields an even integer equal to an odd integer. Thus increasing the degree bound in a rational-polynomial witness dictionary cannot solve the uniform square-root witness problem. At the concrete parameter $t = 2$, any defined expression built solely from rational constants and rational field operations is also rational, which makes the obstruction even more immediate.

An existing library provider such as a square-root function can of course change the witness language. The claim is only about the specified rational-expression grammar, not about every conceivable Lean term synthesizer. The generic root descriptor extends that grammar without requiring a named closed-form function for every algebraic equation. The quintic witness in the package illustrates this use without assuming a formula in radicals.

9.3 Proof-only and computational providers

A future Lean count theorem can produce an existence proof, from which `Classical.choose` supplies a real witness when noncomputability is acceptable. Such a term is a valid mathematical witness but is not an algorithm for evaluating arbitrary real inputs.

For computational synthesis, the provider should instead expose a root descriptor over exact algebraic inputs and a verified interpretation into $\mathbb{R}$. Generalizing the implemented selector to several fiber variables requires algebraic-number arithmetic over successive extensions, certified coordinate isolation, and a proof that the selected tuple satisfies every source equation. None of those steps is delivered here. An interface should report this distinction explicitly rather than advertise a proof-only choice operation as an executable root solver.

10 Concrete algorithms and cost controls

10.1 Producer and replay pipelines

The main producer follows this sequence:

```
produceFiber(source):
  admit the monic triangular source and fixed variable order
  interpolate the truth indicator on the sign cube
  construct the quotient basis by the prescribed degree bounds
  compute multiplication matrices using a Groebner normal form
  build H1 once
  for each nonzero indicator coefficient c[e]:
    w := product of source atoms q[j]^e[j]
    H := H1 * multiplicationMatrix(w)
    emit e, c[e], H, characteristicPolynomial(H)
```

The replay is deliberately different:

```
checkFiber(source, receipt):
  re-check the source schema and literal monic triangular shape
  compile the source Boolean indicator recursively
  require exactly its nonzero support, in canonical order
  for each required e:
    recompute H[i,j] = trace(multiply(w*b[i]*b[j]))
      by descending-variable substitution and diagonal coefficients
    require exact equality with the supplied H
    recompute trace(H), ..., trace(H^D)
    check all Newton identities for the supplied coefficients
  return the symbolic count circuit, not a Lean proof
```

The certificate supplies no floating eigenvalues, approximate zero tests, guessed multiplicities, or unchecked answer table. Its data are rational polynomials, exponent vectors, and exact matrix coefficients.

10.2 Arithmetic cost rather than a misleading speed guarantee

Let C_{red} denote the cost of a normal-form reduction at the relevant degrees, and count arithmetic operations in $R_0 = \mathbb{Q}[t]$ separately from their bit cost. The direct trace replay performs on the order of rD^3 reductions. Its straightforward Newton replay uses D dense matrix multiplications per query, giving $O(rD^4)$ base-ring operations. These estimates explain the prototype's small dimension cap.

They are not bit-complexity bounds. Sparse products can become dense; parameter degrees

can grow; rational numerator and denominator lengths matter; and $D = \prod_i d_i$ already grows exponentially with the number of fiber variables when degrees are fixed above one. A low-degree but many-variable problem is not necessarily cheap.

For one outer parameter, let L be the degree of the squarefree cover polynomial. There are at most $2L + 1$ real cells, but constructing and isolating the polynomial may dominate runtime. Subresultant remainder sequences and modular arithmetic are appropriate engineering improvements, provided their outputs still replay against exact identities and exact root counts.

10.3 Optimizations with an unchanged acceptance theorem

Compile sparsely. Replace truth-table interpolation in the producer by a memoized recursive indicator compiler or a ternary decision diagram. Keep the existing recursive replay independently specified. The current producer's matrix queries are demand-directed, but its initial Boolean construction still enumerates 3^m sign vectors; this cost must not be hidden in a sparse-query headline.

Cache reduced weights and traces. Distinct sign monomials may reduce to the same quotient element. Exact equality of their normal forms allows reuse of the same Hermite matrix. Precompute the trace functional on the quotient basis; after a normal form is known, its trace becomes a linear combination of cached basis traces. For a faster replay variant, accompany a multiplication table with local reduction identities and prove the table's regular-representation law. Do not replace this law by the much weaker assertion that its matrices satisfy the equations.

Use exact backends where they fit. SymPy supplies the symbolic polynomial and matrix operations used here [8]. Python-FLINT exposes rational-matrix characteristic polynomials and exact arithmetic, making it a candidate accelerator for concrete rational specializations [9]. That interface is not by itself a multivariate symbolic Hermite engine. A symbolic FLINT-backed producer would need an additional polynomial-matrix or modular-interpolation layer; the Newton replay can remain unchanged.

Keep specialization and uniform checking distinct. At one concrete parameter value, exact rational LDL^T or other inertia methods may be cheaper than a uniform characteristic-coefficient circuit. That is a useful separate path. It does not certify the unsampled parameter values. A controller should label these two products differently and never cache a pointwise inertia result as a uniform theorem.

10.4 A safe route beyond literally triangular input

There are two useful extensions that can reuse existing Forge algebraic machinery rather than duplicate it.

First, select a monic triangular subsystem and leave all additional equations as equality atoms in F . The admitted tower supplies a finite containing fiber; the formula filters it back to the original source solution set. This handles some overdetermined systems without any ideal-equivalence certificate.

Second, let an untrusted elimination engine propose a triangular presentation T_i . Require polynomial multiplier identities in *both* directions,

$$T_i = \sum_j a_{ij} f_j, \quad f_j = \sum_i b_{ji} T_i.$$

Exact expansion proves ideal equality, hence equality of the source zero sets at every specialization.

Forge’s existing ideal-multiplier work can supply that acceptance mechanism; the new role is to justify the finite-fiber presentation. If the identities require parameter denominators, their nonzero guards and complementary cases must be proved separately. This normalization frontend is proposed, not implemented in the package.

Neither extension makes a positive-dimensional source finite by decree. If no finite containing tower or equivalent finite presentation can be certified, the worker declines the problem.

11 A concrete Lean integration plan

The mathematical contribution should enter Forge as a new worker and a small proof library, not as a replacement for its controller. The existing `grind` infrastructure is useful for congruence, propagation, and arithmetic around a derived count guard [14]. No modification of its internals is necessary for the first integration.

11.1 The proof obligations are modular

The implementation should separate the following proof modules. These are proposed paths and responsibilities, not files claimed to exist in this package.

Proposed module	Required theorem, stated by its mathematical content
<code>Forge/RealFiber/Signs.lean</code>	The Boolean compiler agrees with the truth indicator on the finite sign cube; its weighted sum agrees with a filtered finite cardinality.
<code>Forge/RealFiber/Triangular.lean</code>	Monic-tower normal forms are unique; the rectangular monomials form a basis; normalization and matrix entries commute with parameter evaluation.
<code>Forge/RealFiber/Hermitic.lean</code>	The weighted trace-form signature equals the Tarski query on distinct real zeros, including nonreduced algebras.
<code>Forge/RealFiber/CharpolySign.lean</code>	Newton replay identifies the characteristic polynomial; a symmetric matrix has real-rooted characteristic polynomial; zero-deleting variations give its inertia difference.
<code>Forge/RealFiber/LineCover.lean</code>	The checked rational isolators contain every root of the cover polynomial; reconstructed signs are correct on all point and open cells.
<code>Forge/RealFiber/RootSelection.lean</code>	A concrete isolator denotes one real root, and checked gcd/root-count facts establish all source atom signs there.
<code>Forge/RealFiber/Bridge.lean</code>	The reified equations and Boolean formula denote the exact original Lean relation under the recorded parameter map.

The hardest missing theorem is the full nonreduced Hermite-signature bridge. A theorem only for separable field extensions does not suffice: the quotient of x^2 is not a field and its trace form is degenerate. This limitation must be visible in the milestone acceptance tests. The initial formalization may instead target a smaller, explicitly labeled squarefree univariate lane, but that would not establish the advertised collision-safe full fragment.

11.2 Verified library footholds, not invented APIs

Current Mathlib documentation supplies several concrete starting points. These were checked in the rolling documentation; their availability and signatures must still be compiled against Forge’s chosen pinned Mathlib revision.

Monic quotients. `AdjoinRoot.powerBasis’` takes a monic polynomial over a commutative ring and supplies its power basis. The same module contains `Polynomial.Monic.free_adjoinRoot` and `Polynomial.Monic.finite_adjoinRoot`. These are the appropriate entry points for an iterated monic tower, rather than an irreducible-field-only construction [10].

Trace matrices. The trace library defines the algebra trace and trace form and relates a basis trace matrix to the matrix of that bilinear form. It also records trace behavior for nilpotent elements. These are useful components, but their existence does not mean that the weighted Artinian-algebra signature theorem has already been assembled [11].

Characteristic polynomials and specialization. `Matrix.charpoly_map` gives the natural map behavior of the characteristic polynomial. The characteristic-coefficient files contain monicity and coefficient facts. They are suitable for identifying the reflected polynomial with the mathematical determinant after parameter evaluation; the complete Newton-recursion replay theorem still requires an explicit implementation or a proved equivalent checker [12].

Real spectrum. `Matrix.IsHermitian.charpoly_eq` expresses the characteristic polynomial as a product over the real eigenvalues, and the same spectrum module relates roots to those eigenvalues. This directly supports the real-rootedness premise that makes the variation formula exact [13].

Root signs and quantified closure. The Isabelle developments provide useful formalization precedents for algebraic root certificates and full real quantifier elimination [6, 7]. They are research references, not Lean libraries that can be imported. Forge’s own ledger explicitly identifies its generic Lean Sturm bridge as unfinished at the audited snapshot. A port or a new proof must be compiled and tested rather than assumed from the presence of a Python Sturm routine [1].

11.3 The new source bridge must prove equivalence

A reifier should return the original relation, a parameter map, a finite-fiber IR, and a proof of

$$\text{OriginalRelation}(\rho, \mathbf{x}) \iff (\mathbf{f}(\text{params}(\rho), \mathbf{x}) = 0 \wedge F(\text{params}(\rho), \mathbf{x})). \quad (18)$$

It must preserve real-variable identity, binder order, powers, rational coefficients, strictness, and every relevant source hypothesis. An unproved syntactic extraction is not sufficient for either positive or negative conclusions.

There is an important asymmetry when a parameter is an opaque expression. Suppose the reifier replaces u^2 by a new real parameter t . A theorem valid for *all* t can safely be specialized to u^2 . But a counterexample at $t = -1$ is not a counterexample to the original source. The source parameter map needs a realizability witness before an outer counterparameter may be exported as a source counterexample. Retaining the known guard $t \geq 0$ helps, but an arbitrary collection of guards need not characterize the map’s exact image.

This is not a reason to give up opaque parameters. It is a reason for the worker’s positive transport rule and negative transport rule to have different premises. The new module should fit these premises into Forge’s existing source-owned evidence discipline, not invent a competing authority system.

11.4 What successful replay should return

The desired theorem has the following shape, written as design notation rather than executable Lean:

```
replay_count_sound(source, certificate, accepted):
  for every real parameter assignment tau,
    interpreted_count_circuit(certificate, tau)
      = cardinality(source_real_fiber_filtered_by_formula(tau))
```

The cardinality side must carry the proved finiteness supplied by the monic tower. A cardinality convention that returns zero for an infinite set is not a substitute for that proof.

The tactic uses this equality and the source bridge to rewrite the original existential or unique-existential goal. When a whole one-parameter statement is present, the line-cover theorem closes the remaining condition. With several free parameters, the worker can return the exact count guard as a residual goal or a proved consequence; existing arithmetic workers may then discharge it. A failed guard proof is not a failure of the count identity.

The first production acceptance must produce an ordinary Lean proof term under the project's existing verification gate. It must not add an axiom named after a successful external run, infer success from an elaborator message alone, or trust a Python Boolean. The Python class called `CheckedFiber` is an ordinary mutable research object, not an unforgeable proof handle.

11.5 Admission and scheduling algorithms

A practical admission pass can enumerate candidate leading variables in each polynomial. A candidate is allowed when its leading coefficient is a nonzero rational constant and its lower coefficients use only parameters and earlier fiber variables. Normalize that constant with an equality proof. For a small number of eliminated variables, search assignments of equations to leading variables and topological orders of their dependency graph. Reject a cyclic or unresolved order unless the certified normalization frontend supplies a valid triangular presentation.

After admission, compute D and an upper bound on indicator support before spawning an expensive producer. Route a goal to this worker when it asks for algebraic existence, uniqueness, a finite-fiber universal property, or a count-dependent consequence. A rational-polynomial witness search that has exhausted its small grammar is another useful trigger; the square-root obstruction explains why repeating that search with larger coefficients may be pointless.

The result can cooperate with other workers in both directions. From a proof that $x^2 = t$ has a real witness, the exact existence guard yields $t \geq 0$. Conversely, a proved parameter sign can settle several characteristic coefficients without branching. The first integration can use ordinary proved lemmas for these transfers. It need not attach a new mutable real-algebraic state to `grind`.

12 A specific Leant/Djex extension

Leant and Djex already devote substantial machinery to source selections, bounded synthesis, and checked candidates. Forge's dedicated review has already incorporated many of those lessons [1, 2, 3]. The additional proposal here is concrete: register an *algebraic refinement provider* whose precondition is a checked count guard and whose output is the exact refined

witness requested by the source.

12.1 A provider that supplies mathematical data with its property

A user may ask for a value of the mathematical type

$$\{x : \mathbb{R} \mid x > 0 \wedge x^5 - x = t\}$$

in a context containing $t \geq 0$. A type-directed search need not enumerate real-valued rational expressions indefinitely. It can submit the polynomial relation and the local parameter guard to Real Fibers. The count theorem proves the subtype nonempty; a selected witness is then returned through either a proof-only choice provider or an executable algebraic-descriptor provider.

A proposed provider response should carry four logically separate items: the exact source relation, the checked count theorem, a selection construction when one is available, and a proof that the selected value has the original refinement. The selection construction is absent when only an existence proof is available. This avoids presenting a root count as a synthesized implementation.

At a higher-order boundary, the provider can synthesize a function such as

$$(t : \mathbb{R}) \mapsto \{x : \mathbb{R} \mid x > 0 \wedge x^5 - x = t\} \quad \text{under the argument refinement } t \geq 0.$$

With `Classical.choose`, this is a proof-backed noncomputable function. With a future algebraic-input API, it can be a computational function on that narrower domain. The mode should be part of provider selection, not inferred from whether an emitted term happens to elaborate.

12.2 What this does not solve in the related projects

The audited related READMEs distinguish accepted fragments from outstanding original indexing and simultaneous-universe composition goals. This worker does not resolve those tasks. Its carrier is the specific mathematical domain $\mathbb{R}$, with a polynomial source bridge. It neither erases universe constraints nor substitutes an easier ordinary-list implementation for a Church-encoded target.

The useful transfer is narrower: a type-directed synthesizer gains a new class of *verified providers* instead of more unstructured term enumeration. The count-based provider must preserve the exact requested source refinement, just as the existing synthesis boundary preserves the candidate's selected types and dictionaries. A provider for a nearby but different equation is not a successful answer.

13 Implementation milestones with acceptance gates

Milestone 1: formal sign-count compilation. Implement the sign-cube algebra and its finite-cardinality theorem. Replay every Boolean fixture in the package, including strict and non-strict atoms and correlated constraints. Acceptance is a generic theorem plus tests against the original formula AST, not 160 unrelated hand-proved examples.

Milestone 2: monic normal forms and matrix identities. Formalize the tower basis and the specialization theorem. Verify the emitted matrices and Newton identities in Lean. Acceptance must include a repeated-root source and a two-variable triangular source; proving only a matrix identity for a manually supplied matrix does not establish the source bridge.

Milestone 3: the signature-to-root-count theorem. Connect the weighted trace matrix to distinct real roots, including nilpotents and complex factors. Required controls are x^2 , $x^2 + 1$, $(x - t)^4$, and the coupled tower $x^2 = t$, $y^2 = x$. A radical-only theorem is an explicitly smaller milestone, not completion of this one.

Milestone 4: root covers and selectors. Formalize complete one-dimensional covers and the gcd-based witness signs. The gate includes the two boundary-only false claims and the algebraic-section-only existential claim. A proof valid only on open sectors does not pass.

Milestone 5: end-to-end Forge and synthesis integration. Add the source reifier, count-guard lowering, and mode-labeled provider. Run a pinned Lean corpus with matched resource budgets against the existing tactics. Report solved goals, proof sizes, elaboration cost, and failures separately. Until this gate is met, there is no supported claim that Real Fibers improves `grind`’s measured solve rate.

This ordering isolates the substantial formal-algebra work rather than hiding it behind a large tactic prototype. The tested Python implementation supplies concrete IRs and counterexamples for each gate, while the article’s proofs explain exactly what must be kernelized.

14 Experiments, controls, and what the numbers mean

The final recorded corpus is `results/run-02`. It was generated with Python 3.13.5, SymPy 1.14.0, and seed 20260915. The implementation is small enough that every stored valid bundle and every stored mutation can be replayed without the symbolic producer. These are component-level experiments, not results from the Forge repository’s own suites.

14.1 Evidence categories

There are 99 valid stored bundles. The 17 universal families contribute 578 rational parameter specializations; the two two-parameter families contribute 98; the concrete random sources contribute 72. Thus standalone replay repeats 748 exact count comparisons. The symbolic identity checks are also rerun: the specialization tests are not being substituted for them.

The final outer verdict distribution is sixteen true and three false across nineteen quantified problems. A false verdict is an expected, checked outcome, not a test failure. The complete cover supplies the exact region or section responsible for it.

14.2 Exact oracle design and common-mode risks

For random univariate sources, the oracle isolates the roots of the squarefree union of the equation and atom polynomials, using SymPy’s root routines rather than matrix signatures. For generated triangular systems, the equations are built from rational linear factors and the expected solution tuples are enumerated directly, with duplicates removed. The matrix-signature fixtures have the form $T^T \text{diag}(d_i)T$, where T is unit lower triangular; their expected signature is $\sum_i \text{sgn } d_i$.

These checks probe different mathematics, but there are shared components. The root oracle and producer both depend on SymPy. The producer and checker share the canonical sparse-polynomial representation and source admission code. The outer producer uses the same cover-product construction and verdict evaluator as replay, although isolation and replay root counting are separate. The tests therefore do not justify a claim of completely independent implementations of every layer.

Category	Number	Acceptance criterion
Boolean formulas	160	Recursive indicators equal independently interpolated indicators.
Sign-cube assignments	4,800	Indicator value equals the source formula’s truth value at every enumerated assignment.
Symmetric matrices	90	Characteristic-sign signature agrees with the known inertia of a unimodular congruence of a diagonal matrix.
Universal parameter problems	17	Complete covers give the expected verdict; 15 true and 2 false.
Existential parameter problems	2	One impossible count and one algebraic-section-only witness give the expected opposite verdicts.
Total parameter cells	113	All sections and sectors are reconstructed, including unbounded tails.
Two-parameter families	2	Uniform identities replay; 98 rational parameter pairs agree with an exact root oracle.
Random univariate sources	48	Counts agree with exact SymPy root isolation/-counting.
Random triangular sources	24	Counts agree with independently enumerated rational solution tuples.
Algebraic witnesses	6	Source isolation and every atom sign replay exactly.
Invalid mutations	973	Each altered source/certificate pair is rejected.
Unit tests	25	Standard-library suite passes, including unsupported-input and API-contract controls.

Table 3: Distinct evidence categories. They are not added into a single “proof count”.

14.3 Representative symbolic certificates

Table 4 reports selected artifacts. “Bytes” is the compact JSON size of the certificate only, not of the source problem or pretty-printed evidence bundle. “Queries” counts the nonzero sign-indicator support, not the producer’s truth-table construction work.

Family	D	Queries	$\deg P$	Cells	Bytes
Nonnegative square root	2	3	2	5	617
Positive shifted square root	2	2	5	11	758
Coupled $x^2 = t, y^2 = x, y > 0$	4	2	3	7	689
Positive quadrant, $x^2 = y^2 = t$	4	4	6	5	1,174
Correlated contradictory signs	2	4	0	1	686
Repeated root $(x - t)^4$	4	2	7	3	885
Algebraic-boundary false claim	2	2	4	9	647
Positive quintic root for $t \geq 0$	5	2	25	15	1,187

Table 4: Measured artifacts from the final run. The cover need not be minimal.

The quintic certificate establishes (1) through the complete parameter cover. Its larger cover polynomial contains twenty-five complex roots counted by degree, but only seven distinct real roots, hence fifteen real cells. This is one example where degree, real boundary count, quotient dimension, and admissible fiber count are four different quantities.

The two-parameter cases are the depressed cubic $x^3 + tx + u = 0$ with the condition $x > 0$, and

the collision family $x^2 = (t - u)^2$ with the same condition. The uniform certificate construction works without choosing a parameter cell first. Only rational specialization checks, not a complete two-dimensional parameter closure, were run for these examples.

14.4 Ablations that demonstrate a capability distinction

Remove section checks. Both universal boundary-only false claims have true conditions on every open sector. An open-sector-only test would accept them. The full replay rejects their claimed universal truth and records the exact sections that fail. This ablation is evaluated on the same cover, so it isolates the missing boundary reasoning rather than changing the problem.

Remove mixed sign queries. The correlation fixture has one root satisfying each positive marginal and zero satisfying their conjunction. The four mixed queries reconstruct zero. A procedure using only the two marginal counts cannot distinguish this behavior from a different joint sign distribution with a common positive point. This is an information loss, not a low timeout.

Restrict witnesses to rational arithmetic. The $\sqrt{2}$ descriptor succeeds while a rational-valued witness grammar cannot represent the answer. This is a mathematical expressivity comparison, not a benchmark run of Forge’s existing witness search. No timing or solve-rate result is attributed to that existing implementation.

Replace distinct-root counting by dimension. The empty-real-fiber and repeated-root controls reject that interpretation immediately. A quotient dimension of two can mean zero, one, or two distinct real points. These fixtures exercise the nonreduced and complex contributions of the Hermite theorem.

14.5 Mutation coverage

Mutations change a Hermite-matrix entry, a characteristic coefficient, or a formula weight; omit a required query or a root interval; collapse an isolator; flip a quantified verdict; change the source equation or inequality strictness; or violate the rational/schema format. The final run retains all 973 altered input pairs and their rejection reasons.

A mutation count is not a count of independent vulnerabilities. Many are generated instances of the same corruption family. Nor does rejection of these cases prove hostile-input safety. The implementation has useful size and arithmetic caps but is not a hardened parser, a resource-isolated service, or a formally verified Python interpreter. Uncaught malformed-input errors would need to be treated as failure by a production wrapper, never as a theorem result.

14.6 Standalone replay and retained corrections

The standalone command runs with `python -S`, which disables site-package initialization. Its import guard forbids SymPy, NumPy, FLINT, and the producer. It rechecks all 99 valid bundles, all 973 mutations, the Boolean fixtures, and the matrix fixtures. The recorded replay passed with 748 count comparisons, 4,800 truth assignments, and 90 signature matrices.

The final experiment generation and checks took about 35.595 seconds in this environment; standalone corpus replay took about 26.395 seconds. These are whole-script wall times, including many repeated checks and mutation replays. They are not Lean elaboration times, not a fair solver comparison, and not a general performance promise. The producer timings in the raw per-family records include internal acceptance checks performed by the outer producer.

An earlier successful run covered sixteen universal families before the quintic and the two existential cases were added. Its corpus and logs are retained as **results/run-01**; no counts from that run are added to the final totals. A separate first unit-test attempt had two errors because the tests passed raw coefficients containing ± 2 to an API that requires coefficient *signs*. The checker correctly rejected those inputs. The tests were corrected, a misuse-rejection regression was added, and the initial log remains available. No checker acceptance condition was weakened.

15 Reproducing and extending the package

From the archive root, the following commands replay the final corpus and run the small unit suite without third-party packages:

```
python -S prototype/bin/replay.py results/run-02
cd prototype
python -S -m unittest discover -s tests -v
```

Generation requires SymPy 1.14.0. From the archive root:

```
python -m pip install -r prototype/requirements.txt
python prototype/bin/run_experiments.py --output reproduced-results
```

The generator refuses to overwrite a nonempty output directory. There is no network operation in the producer or replay code. Installing the declared dependency is separate from running them.

A small API example, run with **prototype** on the Python import path, is:

```
import sympy as s
from real_fibers.producer import make_problem, produce
from real_fibers.checker import verify

x, t = s.symbols("x t")
source = make_problem(
    [t], [x], [x*x - t], [x], ["atom", 0, "ge"]
)
certificate = produce(source)
checked = verify(source, certificate)
assert checked.count([-1]) == 0
assert checked.count([0]) == 1
assert checked.count([2]) == 1
```

For a complete one-parameter theorem, the outer source also records its parameter atoms, a Boolean condition combining their signs with count comparisons, and its outer quantifier. The artifact **results/run-02/certificates/quintic_positive.json** is a worked template. The source and certificate are separate fields; a service integrating the code must continue to take the source from its own trusted request rather than trust an attacker-supplied replacement source alongside a receipt.

The LaTeX article builds independently of the Python dependency:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error forge-real-fibers.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-real-fibers.tex
```

All article section sources are included. External papers, repository code, and fonts are not redistributed.

16 Conclusion

The strongest proposed addition is not another general search controller. It is a source-bound, specialization-safe algebraic service with an exact meaning: count the distinct real points in a finite nonlinear fiber that satisfy the requested Boolean condition. Goal-directed sign indicators reduce the requested Tarski queries; a monic tower makes the quotient basis valid everywhere; Hermite signatures retain multiplicities correctly without counting them as separate points; and characteristic-polynomial signs remove the need to pretend that symbolic pivots never vanish.

One-parameter closure then turns this local algebraic service into a genuine quantifier-alternation capability, with exact boundary counterexamples and witnesses. Concrete algebraic descriptors extend synthesis beyond rational-expression grammars. The delivered prototypes exercise these paths, but the remaining Lean trace-form theorem, source bridge, and end-to-end tactic integration are substantial work and are explicitly separated from the tested artifacts.

The next useful merge into Forge is therefore a small formal sign-count library and a precise source IR, followed by the monic-tower and trace-form bridges. Those components would give the existing controller and the related synthesis projects a materially new kind of result to use, while preserving their current evidence discipline.

A Certificate schema and implementation map

A polynomial in v variables is serialized as a lexicographically increasing list of pairs $(e, [a, b])$, where $e \in \mathbb{N}^v$, $a/b \in \mathbb{Q}$, $b > 0$, and $\gcd(|a|, b) = 1$. Zero coefficients are omitted. Duplicate or out-of-order monomials are rejected. Variables are ordered as all parameters followed by all fiber variables. Matrix rows and columns use the rectangular basis in the same fixed product order.

The main source schema is **forge-real-fiber-v1**. It records the parameter count, leading degrees, source equations, atom polynomials, and Boolean formula. The receipt schema **hermite-charpoly-v1** has one row per required indicator monomial. A row contains its exponent vector, rational weight, Hermite matrix, and characteristic coefficient list in descending degree order, including the leading one. Missing, reordered, or extra rows are rejected.

The outer source schema is **outer-one-parameter-v1**; its receipt is **outer-sign-cover-v1**. It contains the fiber receipt, ordered open rational root isolators, and a Boolean claim. The quantifier and condition are owned by the outer source. The witness receipt is **algebraic-witness-v1**, containing only its rational interval; the polynomial and property remain in the source.

Implementation file	Responsibility
<code>real_fibers/exact.py</code>	Canonical rational sparse polynomials, matrix operations, sign variations, schema exceptions and caps.
<code>real_fibers/logic.py</code>	Boolean AST validation, evaluation, and recursive sign-indicator compilation.
<code>real_fibers/checker.py</code>	Source admission, triangular reduction, trace matrices, Newton replay and count-circuit evaluation.

Implementation file	Responsibility
<code>real_fibers/sturm.py</code>	Rational univariate arithmetic, gcd, squarefree root union and distinct-root counts.
<code>real_fibers/outer.py</code>	Source-owned quantified conditions, complete boundary coverage and section/sector reconstruction.
<code>real_fibers/witness.py</code>	Concrete univariate source-root isolation and gcd-based atom signs.
<code>real_fibers/producer.py</code>	Untrusted SymPy interpolation, quotient computations, characteristic polynomials and root-isolation proposals.
<code>bin/run_experiments.py</code>	Reproducible source generation, exact oracles, artifact emission and structured mutations.
<code>bin/replay.py</code>	Standalone no-producer replay, strict duplicate-key JSON reading, fixture checks and outcome summary.
<code>tests/test_core.py</code>	Twenty-five focused standard-library unit tests.

B Research extensions kept separate from current acceptance

A useful next research direction is a *stratified finite presentation*: cover parameter space by proved guards, attach a basis valid on each guard, and prove complete guard coverage, including loci where the quotient dimension changes. This can widen the source language substantially, but it is more than replacing one normal-form call. Both source equivalence and specialization validity must be certified on every stratum.

A second direction is shared count compilation for several formulas on the same fiber. Their indicator supports can share trace queries and characteristic coefficients, producing a vector of counts for one matrix workload. The current API checks one formula per certificate; this sharing requires a new explicit multi-output schema, not reassociation of existing receipts by a presentation-only key.

A third direction is finite-fiber counting as an inverse-function provider. An equation $p(t, x) = 0$ with a checked unique-root region defines an implicit branch without demanding a closed-form expression. The count theorem supplies the branch's uniqueness region; regularity, derivatives, and numerical evaluation require additional theorems and algorithms. Those analytic claims should not be inferred from existence and uniqueness alone.

These extensions are deliberately not included in the implementation totals. They identify concrete ways to grow the new capability after its current mathematical and source bridges are kernel-checked.

References

- [1] Vladimir Reshetnikov. *Forge repository*. Audited snapshot 58ea206bd7ad501b930add568151217bcc7f82a2. README; docs/STATUS.md; docs/IDEAS-FROM-LEANT-DJEX.md; article/sections/07-nonlinear.tex; article/sections/B-provenance.tex; lean/Forge/Design/Contracts.lean. <https://github.com/VladimirReshetnikov/Forge/tree/58ea206bd7ad501b930add568151217bcc7f82a2>.
- [2] Vladimir Reshetnikov. *Leant repository*. README and current priority descriptions; snapshot 6bf05ad78c467989e68290f2d08bbbed40802d485. <https://github.com/VladimirReshetnikov/Leant/tree/6bf05ad78c467989e68290f2d08bbbed40802d485>.
- [3] Vladimir Reshetnikov. *Djex repository*. README, current supported fragments and priority descriptions; snapshot e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5. <https://github.com/VladimirReshetnikov/Djex/tree/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5>.
- [4] Huu Phuoc Le and Mohab Safey El Din. Solving parametric systems of polynomial equations over the reals through Hermite matrices. *Journal of Symbolic Computation* 112 (2022), 25–61. Preprint arXiv:2011.14136. <https://arxiv.org/abs/2011.14136>.
- [5] Louis Gaillard and Mohab Safey El Din. Solving parameter-dependent semi-algebraic systems. *ISSAC 2024*. DOI: 10.1145/3666000.3669718. Revised preprint arXiv:2402.07782v2 (2025). <https://arxiv.org/html/2402.07782v2>.
- [6] Wenda Li, Grant Olney Passmore, and Lawrence C. Paulson. Deciding univariate polynomial problems using untrusted certificates in Isabelle/HOL. *Journal of Automated Reasoning* 62 (2019), 69–91. DOI: 10.1007/s10817-017-9424-6. Preprint arXiv:1506.08238. <https://arxiv.org/html/1506.08238v2>.
- [7] Katherine Kosaian, Yong Kiam Tan, and André Platzer. A first complete algorithm for real quantifier elimination in Isabelle/HOL. *CPP 2023*. DOI: 10.1145/3573105.3575672. <https://symbolaris.com/pub/QE-HybridMultivariate.pdf>.
- [8] SymPy development team. *Polynomial manipulation and matrix documentation*. The prototype uses SymPy 1.14.0; current documentation consulted for exact polynomial, root-isolation, Gröbner, and characteristic-polynomial interfaces. <https://docs.sympy.org/latest/modules/polys/reference.html> and <https://docs.sympy.org/latest/modules/matrices/matrices.html>.
- [9] Python-FLINT contributors. *Matrices over rational numbers*, including fmpq_mat.charpoly. Candidate accelerator, not a dependency of the delivered implementation. https://python-flint.readthedocs.io/en/latest/fmpq_mat.html. See also the underlying FLINT interface: https://flintlib.org/doc/fmpq_mat.html.
- [10] Mathlib contributors. *Mathlib.RingTheory.AdjoinRoot*. Current documentation, including monic power bases and freeness of monic quotients. https://leanprover-community.github.io/mathlib4_docs/Mathlib/RingTheory/AdjoinRoot.html.
- [11] Mathlib contributors. *Mathlib.RingTheory.Trace.Basic*. Algebra traces, trace matrices, trace forms, and related lemmas. https://leanprover-community.github.io/mathlib4_docs/Mathlib/RingTheory/Trace/Basic.html.

- [12] Mathlib contributors. *Matrix characteristic polynomials: Basic and Coeff*. Current documentation for characteristic polynomials, map behavior, and coefficient facts. https://leanprover-community.github.io/mathlib4_docs/Mathlib/LinearAlgebra/Matrix/Charpoly/Basic.html and https://leanprover-community.github.io/mathlib4_docs/Mathlib/LinearAlgebra/Matrix/Charpoly/Coeff.html.
- [13] Mathlib contributors. *Mathlib.Analysis.Matrix.Spectrum*. Spectral theorem and characteristic-polynomial factorization for Hermitian matrices. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Analysis/Matrix/Spectrum.html.
- [14] Lean development team. *The grind tactic*. Current Lean language reference. Used to delimit integration responsibilities, not as a benchmark result. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.